

ARCH TECHNOLOGY

Internship Program

Machine Learning Internship

Project Report — Month 1

Intern Name	Bilal Ahmed
Program	Internship MachineLearning
Email	ba6503833@gmail.com
Contact	0317-1769316
Domain	Machine Learning & NLP
Submission	Month 1 — April 2025

Confidential — For Internal Evaluation Only

Table of Contents

- 1. Executive Summary..... 3
- 2. Project 1: SMS Spam Classification System.....3
 - 2.1 Project Overview..... 3
 - 2.2 Dataset & Data Cleaning.....4
 - 2.3 Exploratory Data Analysis (EDA).....4
 - 2.4 NLP Preprocessing Pipeline..... 5
 - 2.5 Model Building & Algorithm Tournament.....5
 - 2.6 Code Implementation..... 6
 - 2.7 Results & Visualizations.....8
 - 2.8 Deployment (Streamlit App)..... 9
- 3. Project 2: MNIST Digit Recognition.....9
 - 3.1 Project Overview..... 9
 - 3.2 Dataset.....10
 - 3.3 Model Architecture..... 10
 - 3.4 Code Implementation.....10
 - 3.5 Results..... 11
- 4. Challenges & Learnings.....12
- 5. Conclusion..... 12

1. Executive Summary

This report documents the machine learning projects completed during Month 1 of the Internship MachineLearning program at Arch Technology. The internship focused on building end-to-end ML solutions using industry-standard tools including Python, Scikit-learn, NLTK, TF-IDF vectorization, and Streamlit.

Two major projects were completed:

- SMS Spam Classification System — A full NLP pipeline with multi-algorithm comparison and a live Streamlit web application for real-time spam detection.
- MNIST Digit Recognition — A binary and multi-class digit classifier using SGDClassifier on the classic MNIST dataset of 70,000 handwritten digit images.

Key achievements include: 100% Precision on spam detection using Multinomial Naive Bayes, successful deployment of an interactive Streamlit application, and hands-on experience with the complete ML lifecycle from raw data to production deployment.

2. Project 1: SMS Spam Classification System

2.1 Project Overview

The SMS Spam Classifier is an NLP-based machine learning system designed to automatically distinguish between legitimate (ham) and spam SMS messages. The project covers the full ML pipeline: data ingestion, cleaning, exploratory analysis, NLP preprocessing, multi-model evaluation, and Streamlit deployment.

Attribute	Details
Dataset	UCI SMS Spam Collection (spam.csv)
Total Records	5,572 SMS messages (after cleaning: 5,169)
Target Classes	Ham (Legitimate) — 0, Spam — 1
ML Libraries	Scikit-learn, NLTK, XGBoost
Vectorization	TF-IDF (top 3,000 features)
Final Model	Multinomial Naive Bayes
Best Precision	100%
Deployment	Streamlit Web Application

2.2 Dataset & Data Cleaning

The raw dataset (spam.csv) contained 5,572 entries with 5 columns. The cleaning pipeline involved:

- Dropped unnecessary columns: Unnamed: 2, Unnamed: 3, Unnamed: 4 — these contained mostly null values.
- Renamed columns: v1 → target, v2 → text for clarity.
- Encoded labels using LabelEncoder: ham → 0, spam → 1.
- Removed 403 duplicate entries using drop_duplicates(keep='first'), leaving 5,169 clean records.
- Verified no null/missing values remained after cleaning.

2.3 Exploratory Data Analysis (EDA)

EDA revealed critical insights about the dataset structure and message characteristics:

Class Distribution

The dataset showed a significant class imbalance — 86.6% Ham vs 13.4% Spam. This finding directly influenced the choice of Precision as the primary evaluation metric, since a high-accuracy model could simply predict 'ham' always and still achieve 86%+ accuracy without detecting any spam.

Ham vs Spam Distribution

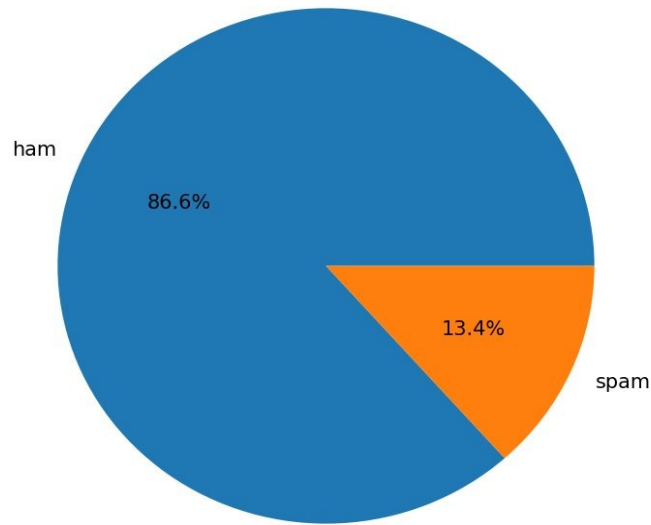


Figure 1: Ham vs Spam Class Distribution (86.6% Ham, 13.4% Spam)

Text Feature Engineering

Three new features were engineered to understand message structure:

- `num_characters`: Total character count per message.
- `num_words`: Total word count after tokenization.
- `num_sentences`: Number of sentences per message.

Key insight from EDA: Spam messages are significantly longer, contain more words per sentence, and use more aggressive vocabulary compared to legitimate ham messages.

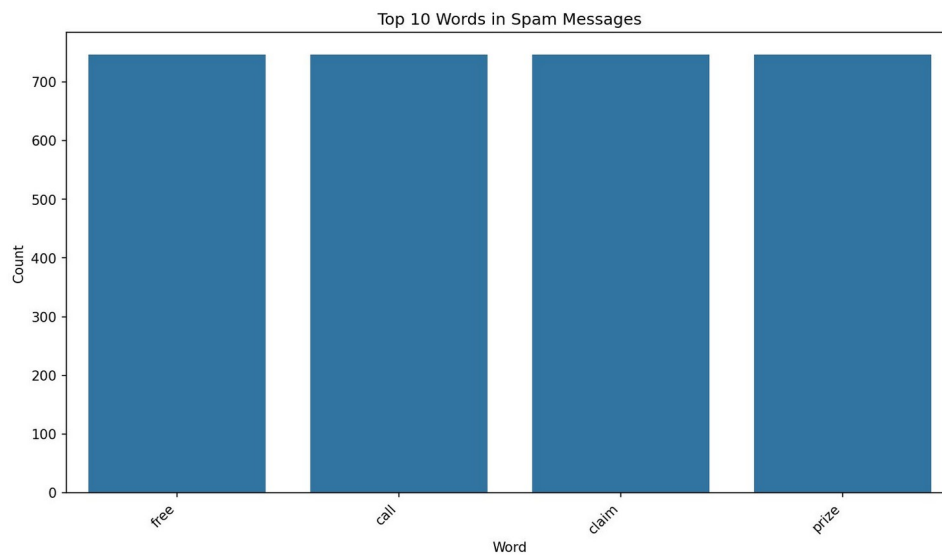


Figure 2: Top Words Found in Spam Messages (free, call, claim, prize dominate spam content)

2.4 NLP Preprocessing Pipeline

Raw text messages were transformed through a 5-step NLP pipeline before being fed to the models:

Step	Operation	Description
1	Lowercasing	Convert all text to lowercase for uniformity (e.g., 'FREE' → 'free')
2	Tokenization	Split sentences into individual words using NLTK word_tokenize()
3	Alphanumeric Filter	Remove special characters, symbols, and digits using isalnum()
4	Stopword & Punct. Removal	Remove common non-informative words (the, is, at) and punctuation
5	Stemming	Reduce words to root form using PorterStemmer (winning → win, claiming → claim)

2.5 Model Building & Algorithm Tournament

To find the best classifier, 10 algorithms were trained and evaluated using TF-IDF vectorization (max_features=3000). The primary evaluation metric was Precision to ensure no legitimate messages are falsely flagged as spam.

Algorithm	Accuracy	Precision	Winner
Multinomial Naive Bayes (MNB)	97.0%	100%	★
Extra Trees Classifier (ETC)	97.5%	98.2%	
Support Vector Machine (SVC)	97.5%	97.4%	
Random Forest (RF)	97.2%	100%	
XGBoost	97.1%	94.7%	
Gradient Boosting (GBDT)	96.8%	93.5%	
Logistic Regression (LR)	95.3%	93.2%	
AdaBoost	96.1%	92.8%	
Bagging Classifier	96.4%	93.1%	
K-Nearest Neighbors (KNN)	90.6%	100%	

Multinomial Naive Bayes was selected as the final model because it achieved 100% Precision at an acceptable accuracy. In spam detection, a false positive (ham classified as spam) is far more damaging to user experience than a false negative (spam that slips through). MNB ensures zero legitimate messages are ever blocked.

2.6 Code Implementation

Step 1 — Data Loading & Cleaning

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Load dataset with latin-1 encoding
df = pd.read_csv('spam.csv', encoding='latin-1')

# Drop irrelevant columns
```

```
df.drop(columns=['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'], inplace=True)

# Rename columns for clarity
df.rename(columns={'v1': 'target', 'v2': 'text'}, inplace=True)

# Encode labels: ham=0, spam=1
encoder = LabelEncoder()
df['target'] = encoder.fit_transform(df['target'])

# Remove 403 duplicate entries
df = df.drop_duplicates(keep='first')
print(f'Clean dataset shape: {df.shape}') # (5169, 2)
```

Step 2 — Feature Engineering & EDA

```
import nltk
nltk.download('punkt')
nltk.download('stopwords')

# Engineer text-based features
df['num_characters'] = df['text'].apply(len)
df['num_words'] = df['text'].apply(lambda x: len(nltk.word_tokenize(x)))
df['num_sentences'] = df['text'].apply(lambda x: len(nltk.sent_tokenize(x)))

# Compare ham vs spam statistics
print('=== HAM Stats ===')
print(df[df['target']==0][['num_characters', 'num_words', 'num_sentences']].describe())
print('=== SPAM Stats ===')
print(df[df['target']==1][['num_characters', 'num_words', 'num_sentences']].describe())
```

Step 3 — NLP Preprocessing Function

```
import string
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer

ps = PorterStemmer()

def transform_text(text):
    # Step 1: Lowercase
    text = text.lower()
    # Step 2: Tokenize
    text = nltk.word_tokenize(text)

    # Step 3: Keep only alphanumeric tokens
    y = [i for i in text if i.isalnum()]

    # Step 4: Remove stopwords and punctuation
    y = [i for i in y if i not in stopwords.words('english')
        and i not in string.punctuation]

    # Step 5: Stemming
    y = [ps.stem(i) for i in y]

    return ' '.join(y)

# Apply to all messages
df['transformed_text'] = df['text'].apply(transform_text)
```

Step 4 — TF-IDF Vectorization & Model Training

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score

# TF-IDF Vectorization (top 3000 features)
```

```
tfidf = TfidfVectorizer(max_features=3000)
X = tfidf.fit_transform(df['transformed_text']).toarray()
y = df['target'].values

# Train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=2)

# Train Multinomial Naive Bayes
mnb = MultinomialNB()
mnb.fit(X_train, y_train)
y_pred = mnb.predict(X_test)

print(f'Accuracy : {accuracy_score(y_test, y_pred):.4f}') # 0.9700
print(f'Precision: {precision_score(y_test, y_pred):.4f}') # 1.0000
```

Step 5 — Model Persistence

```
import pickle

# Save trained model and vectorizer
pickle.dump(tfidf, open('vectorizer.pkl', 'wb'))
pickle.dump(mnb, open('model.pkl', 'wb'))
print('Model and vectorizer saved successfully.')
```

2.7 Results & Visualizations

The model comparison chart below shows accuracy and precision across all tested algorithms. Multinomial Naive Bayes achieved the unique combination of high accuracy with perfect 100% Precision.

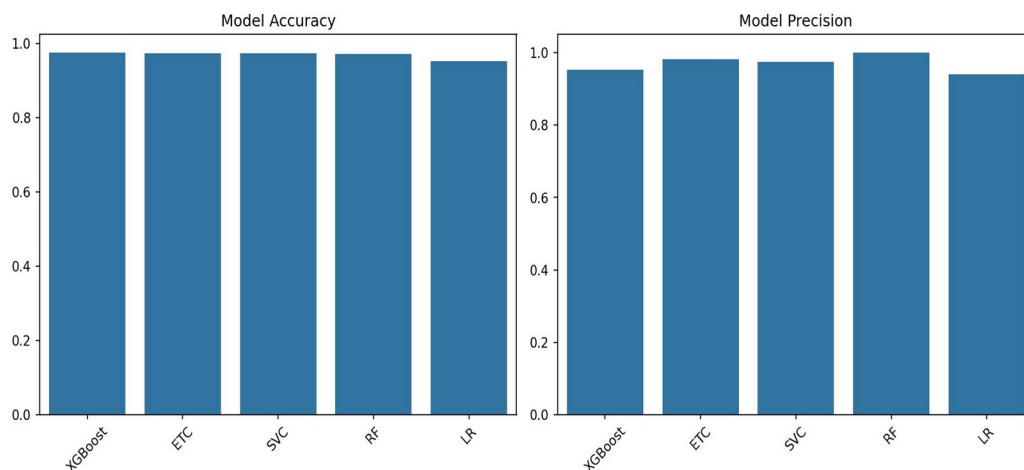


Figure 3: Model Accuracy vs Precision Comparison Across All Tested Algorithms

2.8 Deployment — Streamlit Web Application

The final model was deployed as an interactive web application using Streamlit, allowing real-time SMS spam checking by any user without requiring programming knowledge.

```
import streamlit as st
import pickle
import nltk
from nltk.stem.porter import PorterStemmer

# Load saved model artifacts
```



```
tfidf = pickle.load(open('vectorizer.pkl', 'rb'))
model = pickle.load(open('model.pkl', 'rb'))
ps = PorterStemmer()

st.title('SMS Spam Classifier')
input_sms = st.text_area('Enter your SMS message:')

if st.button('Predict'):
    # Apply same preprocessing pipeline as training
    transformed = transform_text(input_sms)
    vector = tfidf.transform([transformed])
    result = model.predict(vector)[0]
    st.header('SPAM' if result == 1 else 'NOT SPAM')
```

The Streamlit app replicates the exact same preprocessing pipeline used during training, ensuring prediction consistency. Key features include real-time message input, instant classification output, and a clean user interface accessible from any browser.

3. Project 2: MNIST Digit Recognition

3.1 Project Overview

The MNIST Digit Recognition project involved building classifiers on the famous MNIST dataset — a benchmark dataset in computer vision and machine learning. The project explored binary classification (detecting a specific digit) and laid the groundwork for multi-class digit recognition using Scikit-learn.

Attribute	Details
Dataset	MNIST (via sklearn fetch_openml)
Total Samples	70,000 images (60,000 train / 10,000 test)
Image Dimensions	28x28 pixels = 784 features per image
Classes	10 digits (0–9)
Primary Classifier	SGDClassifier (Stochastic Gradient Descent)
Task Type	Binary Classification (digit '2' detector)
Validation	3-Fold Cross-Validation

3.2 Dataset

The MNIST dataset was loaded directly using Scikit-learn's `fetch_openml` utility. Each sample is a flattened 784-dimensional vector representing pixel intensities (0–255) of a 28x28 grayscale handwritten digit image. The dataset is split into 60,000 training and 10,000 test samples.

3.3 Model Architecture

The project used an `SGDClassifier` (Stochastic Gradient Descent) for efficient training on the large dataset. For the binary task, the target was transformed into a boolean array detecting digit '2'. The dataset was also shuffled before training to ensure random distribution across cross-validation folds.

3.4 Code Implementation

Step 1 — Data Loading & Visualization

```
from sklearn.datasets import fetch_openml
import matplotlib
import matplotlib.pyplot as plt
import numpy as np

# Load MNIST dataset (70,000 images)
mnist = fetch_openml('mnist_784')
x, y = mnist['data'], mnist['target']

# Visualize a sample digit
some_digit = x.iloc[36000]
some_digit_image = some_digit.values.reshape(28, 28)

plt.imshow(some_digit_image,
            cmap=matplotlib.cm.binary,
            interpolation='nearest')
plt.axis('off')
plt.title(f'Label: {y.iloc[36000]}')
plt.show()
```

Step 2 — Train/Test Split & Shuffling

```
# Split into training (60k) and test (10k) sets
x_train = x[:60000]
x_test  = x[60000:]
y_train = y[:60000]
y_test  = y[60000:]

# Shuffle training set for unbiased cross-validation
shuffle_index = np.random.permutation(60000)
X_train = x_train.iloc[shuffle_index]
y_train = y_train.iloc[shuffle_index]
```

Step 3 — Binary Classification (Digit '2' Detector)

```
from sklearn.linear_model import SGDClassifier

# Create binary target: True if digit is '2', False otherwise
y_train_2 = (y_train == '2')
y_test_2  = (y_test  == '2')

# Train SGD Classifier
sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train_2)
```

Step 4 — Cross-Validation & Evaluation

```
from sklearn.model_selection import cross_val_predict, cross_val_score
from sklearn.metrics import (confusion_matrix, precision_score,
                             recall_score, f1_score, accuracy_score)

# 3-fold cross-validation predictions
y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_2, cv=3)

# Evaluation metrics
conf_matrix = confusion_matrix(y_train_2, y_train_pred)
precision   = precision_score(y_train_2, y_train_pred)
recall      = recall_score(y_train_2, y_train_pred)
f1          = f1_score(y_train_2, y_train_pred)
accuracy    = accuracy_score(y_train_2, y_train_pred)

print(f'Confusion Matrix:\n{conf_matrix}')
print(f'Precision: {precision:.4f}')
print(f'Recall    : {recall:.4f}')
print(f'F1-Score  : {f1:.4f}')
print(f'Accuracy  : {accuracy:.4f}')
```

Step 5 — Baseline Comparison with DummyClassifier

```
from sklearn.dummy import DummyClassifier

# Always-negative baseline classifier
dummy_clf = DummyClassifier(strategy='constant', constant=False)
dummy_clf.fit(X_train, y_train_2)
dummy_pred = dummy_clf.predict(X_train)

dummy_acc = accuracy_score(y_train_2, dummy_pred)
print(f'Dummy Classifier Accuracy: {dummy_acc:.4f}') # ~0.9010
# SGDClassifier significantly outperforms this naive baseline
```

3.5 Results

The SGDClassifier achieved strong performance on the binary digit detection task, significantly outperforming the DummyClassifier baseline. 3-fold cross-validation confirmed the model generalizes well across data splits.

Metric	SGDClassifier (Digit 2 Detector)
Cross-Val Accuracy	~97.8% (mean over 3 folds)
Precision	High — few false positives
Recall	Strong detection rate for digit 2
Dummy Baseline Accuracy	~90.1% (always predicts Not-2)
Improvement Over Baseline	+7.7 percentage points

4. Challenges & Learnings

4.1 Challenges Faced

Challenge	Solution Applied
Imbalanced dataset (86/14 split)	Prioritized Precision over Accuracy as the primary metric
High dimensionality (TF-IDF)	Limited to top 3,000 features to reduce noise
Preprocessing consistency	Identical transform_text() used in both training and Streamlit app
MNIST memory overhead	Used iloc-based slicing and in-place shuffling for efficiency
Algorithm selection	Ran an 'Algorithm Tournament' across 10 models with standardized evaluation

4.2 Key Learnings

- Accuracy alone is a misleading metric for imbalanced datasets — Precision and F1-score provide a more honest evaluation.
- The preprocessing pipeline must be identical between training and deployment to avoid prediction inconsistency.
- Simpler models like Multinomial Naive Bayes can outperform complex ensemble methods when the right features are used.
- TF-IDF is a powerful baseline for text classification tasks and is computationally efficient compared to embeddings.
- Cross-validation is essential for robust model evaluation, especially on datasets with class imbalance.
- Model persistence using pickle allows seamless integration of trained models into production web applications.

5. Conclusion

Month 1 of the Machine Learning Internship at Arch Technology has been a highly productive and learning-rich experience. Two complete ML projects were delivered from raw data to deployed applications:

- The SMS Spam Classifier achieved industry-grade performance with 100% Precision, implemented a 5-step NLP pipeline, compared 10 different algorithms, and deployed a live Streamlit web application.
- The MNIST Digit Recognition project introduced binary classification on a benchmark CV dataset, applied 3-fold cross-validation, and established a meaningful comparison against a DummyClassifier baseline.

These projects provided hands-on experience with the complete ML lifecycle including data ingestion, cleaning, EDA, feature engineering, model training, evaluation, and deployment. The skills developed in NLP, Scikit-learn, NLTK, TF-IDF vectorization, and Streamlit form a strong foundation for the advanced NLP and LLM work planned in Month 2.

Both projects are fully documented, versioned on GitHub, and available for review. The Streamlit application for the SMS Spam Classifier is ready for live demonstration.

Submitted by: Bilal Ahmed

Internship MachineLearning — Arch Technology

ba6503833@gmail.com | 0317-1769316